

CS 660: Data Mining For Decision Making

Lecture 21 (Week 10)

Varun Dutt

School of Computing and Electrical Engineering
School of Humanities and Social Sciences
Indian Institute of Technology Mandi, India



Scaling the Heights

Mining Frequent Items List: Association Rules

In data mining, association rule learning is a popular and well researched method for discovering interesting relations between variables in large databases. Piatetsky-Shapiro describes analyzing and presenting strong rules discovered in databases using different measures of interestingness. Based on the concept of strong rules, Agrawal introduced association rules for discovering regularities between products in large scale transaction data recorded by point-of-sale (POS) systems in supermarkets. For example, the rule $\{\text{onion, potatoes}\} \Rightarrow \{\text{burger}\}$ found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, he or she is likely to also buy burger. Such information can be used as the basis for decisions about marketing activities such as, e.g., promotional pricing or product placements. In addition to the above example from market basket analysis association rules are employed today in many application areas including Web usage mining, intrusion detection and bioinformatics.

In computer science and data mining, Apriori is a classic algorithm for learning association rules. Apriori is designed to operate on databases containing transactions (for example, collections of items bought by customers, or details of a website frequentation). Other algorithms are designed for finding association rules in data having no transactions (Winepi and Minepi), or having no timestamps (DNA sequencing).

Definitions in Association Rule Mining

Following the original definition by Agrawal the problem of association rule mining is defined as:

Let $I = \{i_1, i_2, \dots, i_n\}$ be a set of n binary attributes called *items*. Let $D = \{t_1, t_2, \dots, t_n\}$ be a set of transactions called the *database*. Each transaction in D has a unique transaction ID and contains a subset of the items in I . A *rule* is defined as an implication of the form $X \rightarrow Y$ where $X, Y \subseteq I$ and $X \cap Y = \emptyset$. The sets of items (for short *itemsets*) X and Y are called *antecedent* (left-hand-side or LHS) and *consequent* (right-hand-side or RHS) of the rule respectively.

To illustrate the concepts, we use a small example from the supermarket domain. The set of items is $I = \{\text{milk}, \text{bread}, \text{butter}, \text{beer}\}$ and a small database containing the items (1 codes presence and 0 absence of an item in a transaction) is shown in the table below. An example rule for the supermarket could be $\{\text{milk}, \text{bread}\} \Rightarrow \{\text{butter}\}$ meaning that if milk and bread is bought, customers also buy butter.

Note: this example is extremely small. In practical applications, a rule needs a support of several hundred transactions before it can be considered statistically significant, and datasets often contain thousands or millions of transactions.

Transaction ID	milk	Bread	butter	beer
1	1	1	0	0
2	0	1	1	0
3	0	0	0	1
4	1	1	1	0
5	0	1	0	0

Example Dataset

Transaction ID	milk	Bread	butter	beer
1	1	1	0	0
2	0	1	1	0
3	0	0	0	1
4	1	1	1	0
5	0	1	0	0

6	1	0	0	0
7	0	1	1	1
8	1	1	1	1
9	0	1	0	1
10	1	1	0	0
11	1	0	0	0
12	0	0	0	1
13	1	1	1	0
14	1	0	1	0
15	1	1	1	1

Concept of Support, Confidence, Lift, and Conviction

To select interesting rules from the set of all possible rules, constraints on various measures of significance and interest can be used. The best-known constraints are minimum thresholds on support and confidence.

Support

The support $\text{supp}(X)$ of an itemset X is defined as the proportion of transactions in the data set which contain the itemset.

$$\text{supp}(X) = \text{no. of transactions which contain the itemset } X / \text{total no. of transactions}$$

In the example database, the itemset {milk,bread,butter} has a support of $4 / 15 = 0.26$ since it occurs in 26% of all transactions. To be even more explicit we can point out that 4 is the number of transactions from the database which contain the itemset {milk,bread,butter} while 15 represents the total number of transactions.

Confidence

The *confidence* of a rule is defined:

$$\text{conf}(X \rightarrow Y) = \text{supp}(X \cup Y) / \text{supp}(X)$$

For the rule {milk,bread} $\Rightarrow$ {butter} we have the following confidence:

$$\text{supp}(\{\text{milk,bread,butter}\}) / \text{supp}(\{\text{milk,bread}\}) = 0.26 / 0.4 = 0.65$$

This means that for 65% of the transactions containing milk and bread the rule is correct. Confidence can be interpreted as an estimate of the probability $P(Y | X)$, the probability of finding the RHS of the rule in transactions under the condition that these transactions also contain the LHS.

Concept of Support, Confidence, Lift, and Conviction

Lift

The *lift* of a rule is defined as:

$$lift(X \rightarrow Y) = \frac{supp(X \cup Y)}{supp(Y) * supp(X)}$$

The rule {milk,bread} $\Rightarrow$ {butter} has the following lift:

$$supp(\{milk,bread,butter\}) / supp(\{butter\}) \times supp(\{milk,bread\}) = 0.26 / 0.46 \times 0.4 = 1.4$$

Conviction

The *conviction* of a rule is defined as:

$$conv(X \rightarrow Y) = \frac{1 - supp(Y)}{1 - conf(X \rightarrow Y)}$$

The rule {milk,bread} $\Rightarrow$ {butter} has the following conviction:

$$1 - supp(\{butter\}) / 1 - conf(\{milk,bread\} \Rightarrow \{butter\}) = 1 - 0.46 / 1 - 0.65 = 1.54$$

The conviction of the rule $X \Rightarrow Y$ can be interpreted as the ratio of the expected frequency that X occurs without Y (that is to say, the frequency that the rule makes an incorrect prediction) if X and Y were independent divided by the observed frequency of incorrect predictions.

In this example, the conviction value of 1.54 shows that the rule {milk,bread} $\Rightarrow$ {butter} would be incorrect 54% more often (1.54 times as often) if the association between X and Y was purely random chance.

Things to do

- Read Decision Tree Classifiers (Extensions) from Shinghal, 2011 (Chapter 3; Material posted on Moodle)
- Read Chapter 7 from Shinghal, 2011
- Read WEKA from Witten's book about decision trees
- Do Assignment 3

Apriori Algorithm

Association rule generation is usually split up into two separate steps:

1. First, minimum support is applied to find all *frequent itemsets* in a database.
2. Second, these frequent itemsets and the minimum confidence constraint are used to form rules.

While the second step is straight forward, the first step needs more attention.

Finding all frequent itemsets in a database is difficult since it involves searching all possible itemsets (item combinations). The set of possible itemsets is the power set over I and has size $2^n - 1$ (excluding the empty set which is not a valid itemset). Although the size of the powerset grows exponentially in the number of items n in I , efficient search is possible using the *downward-closure property* of support (also called *anti-monotonicity*) which guarantees that for a frequent itemset, all its subsets are also frequent and thus for an infrequent itemset, all its supersets must also be infrequent. Exploiting this property, efficient algorithms (e.g., Apriori and Eclat) can find all frequent itemsets.

Apriori Algorithm Pseudocode

```
procedure Apriori (T, minSupport) { //T is the database and minSupport is the minimum support
    L1 = {frequent items};
    for (k = 2; Lk-1 != ∅; k++) {
        Ck = candidates generated from Lk-1
        //that is cartesian product Lk-1 x Lk-1 and eliminating any k-1 size itemset that is not
        //frequent
        for each transaction t in database do{
            #increment the count of all candidates in Ck that are contained in t
            Lk = candidates in Ck with minSupport
        } //end for each
    } //end for
    return  $\bigcup_k L_k$ ;
}
```

Example

A large supermarket tracks sales data by Stock-keeping unit (SKU) for each item, and thus is able to know what items are typically purchased together. Apriori is a moderately efficient way to build a list of frequent purchased item pairs from this data. Let the database of transactions consist of the sets {1,2,3,4}, {1,2,3,4,5}, {2,3,4}, {2,3,5}, {1,2,4}, {1,3,4}, {2,3,4,5}, {1,3,4,5}, {3,4,5}, {1,2,3,5}. Each number corresponds to a product such as "butter" or "water". The first step of Apriori is to count up the frequencies, called the supports, of each member item separately:

Item	Support
1	6
2	7
3	9
4	8
5	6

We can define a minimum support level to qualify as "frequent," which depends on the context. For this case, let min support = 4. Therefore, all are frequent. The next step is to generate a list of all 2-pairs of the frequent items. Had any of the above items not been frequent, they wouldn't have been included as a possible member of possible 2-item pairs. In this way, Apriori *prunes* the tree of all possible sets. In next step we again select only these items (now 2-pairs are items) which are frequent (the pairs written in bold text):



Questions or Comments?